

Algoritmi e Strutture Dati – 9/1/2024 – Parte A

Esercizio -1 Iscriverti allo scritto entro la scadenza.
In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Si consideri questo algoritmo:

```

int fun(int[][] A, int n)
    return frec(A, 1, 1, n, n)

int frec(int[][] A, int si, int sj, int ei, int ej)
    if si == ei or sj == ej then
        return 0
    else
        int sumSoFar = 0
        int i = ei
        int j = ej
        while i > si and j > sj do
            sumSoFar += A[i][j]
            if A[i][j] < A[i-1][j] then
                i = i - 1
            else
                j = j - 1
        int mi = ⌊(si + ei)/2⌋
        int mj = ⌊(sj + ej)/2⌋
        sumSoFar += frec(A, si, sj, mi, mj)
        sumSoFar += frec(A, mi + 1, sj, ei, mj)
        sumSoFar += frec(A, si, mj + 1, mi, ej)
        sumSoFar += frec(A, mi + 1, mj + 1, ei, ej)
        return sumSoFar

```

Scrivere l'equazione di ricorrenza associata a questo algoritmo e calcolare la complessità computazionale che ne deriva, considerando per semplicità una matrice in input di dimensione $n = 2^k$, con k intero non negativo.

A2 – Tree Repair Shop – Punti ≥ 10

Avete memorizzato un grafo non orientato G che rappresenta un albero non radicato, pronto per essere utilizzato come testcase per il corso di Algoritmi. Il vostro compagno di appartamento, da burlone qual è, vi ha fatto uno scherzo: ha aggiunto un arco e ora il grafo non è più un albero!

Scrivete un algoritmo

repairTree(GRAPH G)

che prenda in input il grafo modificato e lo renda nuovamente un albero di n nodi (non necessariamente quello originale), rimuovendo uno o più archi con l'operazione $G.deleteEdge(x, y)$ e senza inserire nuovi archi.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale. In questo caso, si farà attenzione ad algoritmi che hanno complessità definita da $O()$ oppure da $\Theta()$.

A3 – k -Ripetizioni – Punti ≥ 12

Scrivere un algoritmo

int kpeating(item[] S , int n , int k)

che prenda in input una stringa S contenente n caratteri e un valore intero positivo k e restituisca la lunghezza della più lunga sottostringa contigua in cui tutti i caratteri presenti si ripetono almeno k volte nella sottostringa.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale. Una soluzione $\Omega(n^4)$ verrà valutata 50%. Esistono soluzioni $O(n^3)$ e $O(n^2)$.

Alcuni esempi:

- $S = \text{"baaab"} , k = 3$:
la risposta è 3 (la sottostringa "aaa")
- $S = \text{"babbana"} , k = 2$:
la risposta è 5 (la sottostringa "babba")
- $S = \text{"aabb"} , k = 2$:
la risposta è 4 (tutta la stringa)
- $S = \text{"banana"} , k = 1$:
la risposta è 6 (tutta la stringa)
- $S = \text{"banana"} , k = 3$:
la risposta è 0 (nessuna sottostringa)